# Studio Nx

Backend(FastAPI/Python) 16 — verify_password()

2026-10-12

# Backend(FastAPI/Python) 16 — `verify_password()`

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/core/security.py` |
| 작업 | P1-04 회원가입 (템플릿은 P1-05에 있었지만 15번과 짝이라 함께 구현했습니다, PRD 01 FR-103) |
| 작성 | Claude (AI), 2026-10-07, 사용자 검토 |
| 이전 · 다음 | 15 `hash_password()` · 17 `normalize_email()` |

## 1. 하는 일

입력한 비밀번호가 DB에 저장된 해시(15)와 **맞는지** `True` 또는 `False`로 알려 줍니다.

| 호출 | 결과 |
|---|---|
| `verify_password("Abcdef1!", "$2b$12$Q9xK…")` (가입할 때 쓴 비밀번호) | `True` |
| `verify_password("Abcdef1?", "$2b$12$Q9xK…")` | `False` |

해시를 원래 비밀번호로 **되돌리는 것이 아닙니다.** 해시 안에 들어 있는 소금(15 §1)을 꺼내서, 입력한 비밀번호를 같은 방법으로 다시 계산한 뒤 결과가 같은지 봅니다.

## 2. 언제 쓰나

| 쓰는 곳 | 화면 | 하는 일 |
|---|---|---|
| P1-05 `authenticate()` | home.html 로그인 모달 | 이메일로 찾은 회원의 `password_hash`와 비교합니다. `False`면 401 "Email or password is incorrect."로 응답합니다 |
| P1-09 `check_current_password()` | userInfo.html의 비밀번호 변경과 탈퇴 | 지금 비밀번호를 확인합니다. `False`면 400 "Password is incorrect."로 응답합니다 |

## 3. 코드

**import** (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다 (15번과 같고, 이번에 새로 추가한 것은 없습니다). 이 함수는 **`bcrypt`**만 사용합니다.

**함수**

```
def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("ascii"))
    except ValueError:
        return False
```

### 문법 풀이

| import 문법 | 뜻 |
|---|---|
| `import 모듈` | 모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `bcrypt.checkpw`) |

| 식 | 뜻 |
|---|---|
| `bcrypt.checkpw(비밀번호, 해시)` | 해시 안의 소금으로 비밀번호를 다시 계산해서 같으면 `True`입니다. 둘 다 바이트로 넘깁니다 |
| `plain.encode("utf-8")`, `hashed.encode("ascii")` | 문자열을 바이트로 바꿉니다 (15번과 반대 방향) |
| `try: … except ValueError: return False` | `ValueError`가 나면 오류로 멈추지 않고 `False`를 돌려줍니다 |

| 입력 | `bcrypt.checkpw` | 이 함수 |
|---|---|---|
| 맞는 비밀번호 | `True` | `True` |
| 틀린 비밀번호 | `False` | `False` |
| 72바이트가 넘는 비밀번호 (`"A" * 100`) | **`ValueError`** | `False` |
| 망가진 해시 (`""`, `"x"`) | **`ValueError`** ("Invalid salt") | `False` |

## 4. 흐름

1. 비밀번호와 해시를 바이트로 바꿉니다.
2. bcrypt가 해시에서 소금과 비용을 꺼내 비밀번호를 다시 계산합니다 (약 0.2초).
3. 결과가 같으면 `True`, 다르면 `False`를 돌려줍니다.
4. 계산할 수 없는 입력이면 `False`를 돌려줍니다.

## 5. 왜 이렇게

| 선택 | 이유 |
|---|---|
| `ValueError`를 `False`로 바꿈 | 로그인 입력은 100자까지 받습니다(`LoginRequest`). 비밀번호 규칙(14)은 가입할 때만 검사하므로, 로그인 칸에 긴 글을 넣으면 bcrypt가 오류를 던져 **500**이 납니다. 그런 비밀번호로 가입한 회원은 없으므로 "틀림"이 맞는 답입니다 |
| 망가진 해시도 `False` | DB에 잘못된 값이 있어도 로그인이 500으로 깨지지 않고 "틀림"으로 끝납니다 |
| `==`로 직접 비교하지 않음 | `bcrypt.checkpw`는 결과와 상관없이 비교에 걸리는 시간이 같습니다. 그래서 시간 차이로 해시를 추측하는 공격을 막습니다 |
| 템플릿(P1-05)보다 먼저 구현 | 15번과 짝이라서, 15번의 시험을 "해시 → 비교"로 함께 할 수 있습니다 |

**P1-05에서 알아 둘 점:** 이메일이 없는 회원이면 이 함수를 부르지 않아서 응답이 0.2초 빨라집니다. 그 시간 차이로 "이 이메일은 가입되어 있다"는 사실을 알아낼 수 있습니다. 그래서 `authenticate()` 템플릿에는 회원이 없을

때도 가짜 해시로 이 함수를 한 번 부르라고 적혀 있습니다.

## 6. 시험 (`tests/test_password.py`에 11개를 추가해서 모두 32개)

| 시험 | 확인한 것 |
|---|---|
| 맞는 비밀번호 | `True`입니다 |
| 틀린 비밀번호 5개 | 한 글자 다름, 대소문자 다름, 뒤에 공백, 빈 문자열, 두 번 반복은 모두 `False`입니다 |
| 너무 긴 비밀번호 | `"A" * 100`과 한글 30자(90바이트)는 오류 없이 `False`입니다 |
| 망가진 해시 4개 | `""`, `"x"`, `"not-a-bcrypt-hash"`, `"$2b$12$짧음"`은 오류 없이 `False`입니다 |

전체 시험 108개가 통과했고, `ruff` 검사도 통과했습니다.